

Análise de Algoritmo: Teste de Mesa e Casos de Teste

Documentação de Validação e Qualidade de Código

Objetivo dos testes: Verificar a integridade e a precisão analítica da função sob cenários operacionais comuns, condições de limites de variáveis e stress com valores extremos.

Parte A – Teste de Mesa

O teste de mesa é uma técnica manual utilizada para acompanhar o fluxo de execução de um algoritmo, validando se a lógica produz as saídas esperadas a partir de entradas controladas.

Código Fonte Analisado

```
def calcular_total(preco, quantidade):  
    subtotal = preco * quantidade  
    desconto = subtotal * 0.1  
    total = subtotal - desconto  
    return total  
  
# Execução da chamada de teste  
calcular_total(50, 2)
```

Rastreamento de Memória (Matriz de Estados)

Parâmetro: preco	Parâmetro: quantidade	Variável: subtotal	Variável: desconto	Variável: total (Retorno)
50	2	100	10	90

Passo a Passo Detalhado

A execução interna da função segue a ordem sequencial das instruções:

- Cálculo do Subtotal: **subtotal** = $50 \times 2 = 100$
- Aplicação do Desconto (10%): **desconto** = $100 \times 0.1 = 10$
- Dedução do Valor Total: **total** = $100 - 10 = 90$

Retorno Final da Função: 90

Parte B – Casos de Teste

Abaixo estão descritos os cenários planejados para homologar o comportamento da função sob diferentes perfis de dados.

CASO NORMAL 1

Entrada: `calcular_total(100, 3)`

Processamento Interno:

$subtotal = 100 \times 3 = 300$

$desconto = 300 \times 0.1 = 30$

$total = 300 - 30 = 270$

Resultado Esperado: 270

CASO NORMAL 2

Entrada: `calcular_total(20, 5)`

Processamento Interno:

$subtotal = 20 \times 5 = 100$

$desconto = 100 \times 0.1 = 10$

$total = 100 - 10 = 90$

Resultado Esperado: 90

CASO LIMÍTROFE

Entrada: `calcular_total(50, 0)`

Processamento Interno:

$subtotal = 50 \times 0 = 0$

$desconto = 0 \times 0.1 = 0$

$total = 0 - 0 = 0$

Contexto do Teste: Avalia o comportamento do limite inferior do multiplicador de volume (quantidade nula), garantindo que a multiplicação por zero não gere exceções ou inconsistências.

Resultado Esperado: 0

CASO EXTREMO

Entrada: `calcular_total(1000000, 1000)`

Processamento Interno:

$subtotal = 1.000.000 \times 1.000 = 1.000.000.000$

$desconto = 1.000.000.000 \times 0.1 = 100.000.000$

$total = 1.000.000.000 - 100.000.000 = 900.000.000$

Contexto do Teste: Força o algoritmo a trabalhar com ordens de grandeza massivas (escala de bilhões) para validar a capacidade de representação numérica e precisão decimal da arquitetura.

Resultado Esperado: *900.000.000*

Resumo Consolidado dos Testes

A matriz abaixo sumariza o plano de testes executado para fácil auditoria e conferência técnica:

Tipo de Teste	Invocação (Entrada)	Resultado Esperado
Normal 1	<code>calcular_total(100, 3)</code>	270
Normal 2	<code>calcular_total(20, 5)</code>	90
Limítrofe	<code>calcular_total(50, 0)</code>	0
Extremo	<code>calcular_total(1000000, 1000)</code>	900.000.000